

Học Sâu (Deep Learning)

Chương 9: Các Mẫu Kiến trúc Mạng Tích chập (ConvNet Architecture Patterns)

Đại học Đông Á

July 23, 2026

Nội dung chương

1. Nguyên lý Thiết kế Kiến trúc
2. Batch Normalization (Chuẩn hóa Hàng loạt)
3. Kết nối Thặng dư (Residual Connections)
4. Tích chập Tách rời (Separable Convolution)
5. Thực hành: Xây dựng Mini-Xception
6. Tổng kết

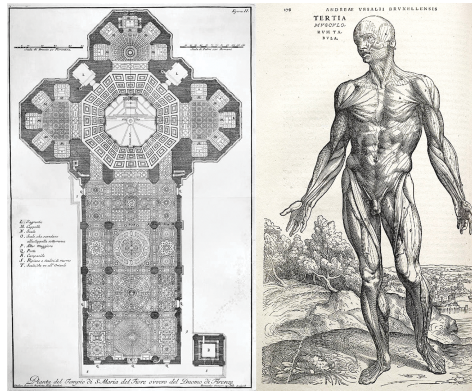
Không gian Giả thuyết (Hypothesis Space)

- **Kiến trúc Mô hình (Model Architecture)** là nghệ thuật lắp ráp các tầng mạng (Layers).
- Cấu trúc này định nghĩa **Không gian Giả thuyết** (Hypothesis Space). Thuật toán Gradient Descent sẽ tìm kiếm bộ tham số (Weights) tối ưu bên trong không gian này.
- Một kiến trúc tốt giống như một bản thiết kế thông minh, giúp đóng gói **Tri thức tiên nghiệm (Prior Knowledge)** của kỹ sư vào mô hình, làm thu hẹp không gian tìm kiếm và tăng tốc độ hội tụ.

Công thức MHR: Modularity, Hierarchy, Reuse

Để quản lý một hệ thống phức tạp, ta dùng nguyên lý **MHR**:

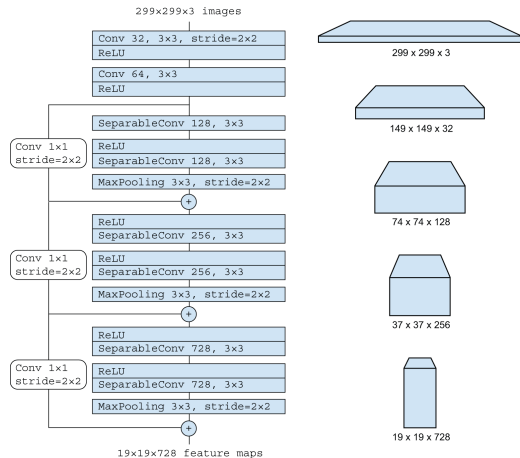
- **Modularity (Mô-đun hóa)**: Đóng gói các lớp thành các khối (Blocks).
- **Hierarchy (Phân cấp)**: Xếp chồng các khối thành tháp (Pyramid).
- **Reuse (Tái sử dụng)**: Tái sử dụng cùng một cấu trúc khối nhiều lần.



Hình 9.1: Hệ thống phức tạp sinh học tuân theo nguyên lý MHR

Tháp Đặc trưng trong Mô hình Xception

- Các mạng CNN hiện đại (như Xception, ResNet) tuân thủ chặt chẽ nguyên lý MHR.
- Cấu trúc Lặp lại: SeparableConv $\rightarrow$ SeparableConv $\rightarrow$ MaxPooling.
- **Tháp không gian:** Kích thước bản đồ (Feature Maps) giảm dần ($299 \times 299 \rightarrow 19 \times 19$).
- **Tháp chiều sâu:** Số lượng bộ lọc (Filters) tăng dần ($32 \rightarrow 728$).



Hình 9.2: Sự phân cấp của Khối đầu vào Xception

Vấn đề: Internal Covariate Shift

- Trong một mạng nhiều tầng, giá trị kích hoạt (Activations) ở một tầng phụ thuộc vào tầng trước nó.
- Khi trọng số của tầng đầu thay đổi trong quá trình huấn luyện (Gradient Descent), **phân phối dữ liệu** ở tầng sau cũng bị xô lệch theo.
- Hiện tượng này gọi là **Internal Covariate Shift**, làm giảm tốc độ hội tụ và khiến mạng sâu rất khó huấn luyện.
- Giải pháp: Chuẩn hóa dữ liệu ngay trong quá trình luân chuyển giữa các tầng.

Toán học của Batch Normalization

Lớp BatchNormalization chuẩn hóa từng Batch dữ liệu trong lúc huấn luyện:

- 1 Tính Kỳ vọng (μ) của Batch: $\mu = \frac{1}{m} \sum_{i=1}^m x_i$
- 2 Tính Phương sai (σ^2) của Batch: $\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2$
- 3 Chuẩn hóa: $\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$
- 4 Chuyển tỷ lệ và dịch chuyển (Scale & Shift): $y_i = \gamma \hat{x}_i + \beta$

Trong đó γ và β là hai tham số học được trong quá trình huấn luyện, đảm bảo mô hình có thể khôi phục lại bất kỳ phân phối nào nếu cần thiết.

Ứng dụng Batch Normalization trong Keras

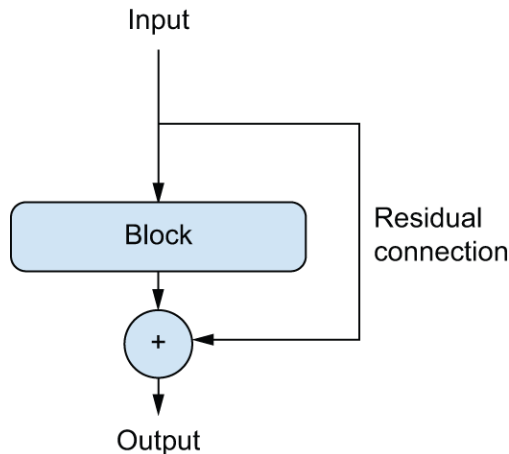
- Vị trí lý tưởng: Thông thường đặt ngay sau tầng Conv2D hoặc Dense và trước khi qua Hàm kích hoạt (Activation).
- **Cơ chế hoạt động kép:**
 - Khi Huấn luyện (`training=True`): Dùng μ và σ^2 của Batch hiện tại. Đồng thời cập nhật trung bình động trượt (Moving Average) cho toàn bộ tập dữ liệu.
 - Khi Suy luận (`training=False`): Dùng giá trị trung bình động (Moving Average) đã tích lũy thay vì tính toán lại, để bảo đảm tính ổn định tuyệt đối cho đầu ra.

Vấn đề: Suy biến Đạo hàm (Gradient Vanishing)

- Nếu bạn liên tục thêm hàng chục tầng Convolutional vào một mô hình (Deep Network), Độ chính xác huấn luyện không tăng mà lại **giảm xuống**.
- Lý do: Thuật toán Backpropagation truyền đạo hàm ngược từ Hàm Loss về lại các tầng đầu tiên thông qua Quy tắc Chuỗi (Chain Rule).
- Phép nhân liên tiếp nhiều số nhỏ hơn 1 khiến đạo hàm giảm mạnh dần về số 0. Mạng bị "bất động", không thể học được.

Giải pháp Kết nối Thặng dư (Residual Connections)

- Phát minh của Kaiming He (2015 - Đạt giải CVPR Best Paper).
- Cung cấp một **đường tắt (Shortcut)** cho thông tin nhảy vọt qua một (hoặc nhiều) lớp không cần tính toán.
- Đạo hàm lúc này sẽ chảy mượt mà qua các đường tắt này về tận đầu mạng mà không bị hao hụt.



Hình 9.3: Cơ chế hoạt động của Khối Residual

Toán học của Khối Residual (ResNet)

Giả sử x là đầu vào của khối, và $F(x)$ là kết quả tính toán qua 2 lớp Convolution.

- Mạng truyền thống (Plain Network): Đầu ra là $y = F(x)$
- Mạng Thặng dư (Residual Network): Đầu ra là $y = F(x) + x$

Thay vì học ánh xạ tổng y , khối Convolution $F(x)$ lúc này chỉ cần học phần **sai khác (thặng dư - Residual)** giữa đầu ra và đầu vào. Việc học "sự thay đổi nhỏ" luôn dễ dàng hơn học "mọi thứ từ đầu".

Giải quyết Bất đồng bộ Kích thước

Khi áp dụng MaxPooling hoặc thay đổi Số lượng Bộ lọc, kích thước của $F(x)$ và x sẽ khác nhau, không thể áp dụng phép cộng trực tiếp.

- Giải pháp: Sử dụng **Linear Projection** (Tích chập 1×1) không kích hoạt (No Activation).
- $y = F(x) + W_s \cdot x$
- Keras: `layers.Conv2D(filters, kernel_size=1, strides=2, padding="same")(x)`

Vấn đề của Tích chập Truyền thống (Conv2D)

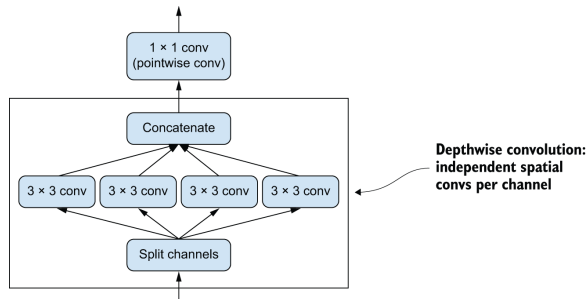
- Tích chập chuẩn thực hiện đồng thời 2 việc trong cùng 1 bước:
 - ① Học mẫu không gian (Spatial Pattern) trên mặt phẳng 2D.
 - ② Tổ hợp thông tin từ nhiều Kênh khác nhau (Cross-channel).
- Sự gán kết này dẫn đến chi phí tính toán cực kỳ lớn, với số lượng tham số là:
 $K^2 \times C_{in} \times C_{out}$.
- Liệu Không gian 2D (Spatial) và Chiều sâu Kênh (Channel) có thực sự phụ thuộc chặt chẽ như vậy?

Khái niệm Separable Convolution

Tách rời phép tính thành 2 pha độc lập:

- 1 **Depthwise Convolution:** Quét từng Kênh một cách riêng biệt (Không giao thoa Kênh).
- 2 **Pointwise Convolution:** Dùng bộ lọc 1×1 để nhào trộn và tổng hợp các kênh.

Giả định: Đặc trưng Không gian (Mắt, Mũi) và Đặc trưng Kênh (Đỏ, Xanh) có tính độc lập cao.



Hình 9.4: Tích chập Tách rời chiều sâu

Ưu điểm vượt trội về Toán học và Tốc độ

- Bằng cách chia rẽ quá trình tính toán, số lượng Tham số (Parameters) được thu gọn:
- Tích chập chuẩn: $K^2 \times C_{in} \times C_{out}$
- Tích chập Tách rời: $K^2 \times C_{in} + C_{in} \times C_{out}$
- **Ví dụ:** Kernel 3×3 , $C_{in} = 64$, $C_{out} = 64$.
 - Lớp Conv2D cần: $3^2 \times 64 \times 64 = 36,864$ tham số.
 - Lớp SeparableConv2D chỉ cần: $(3^2 \times 64) + (64 \times 64) = 576 + 4,096 = 4,672$ tham số.
- → Tiết kiệm gần 90% năng lực tính toán nhưng vẫn duy trì độ chính xác.

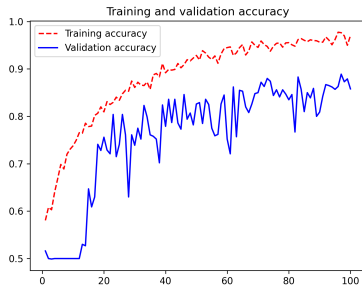
Mô hình Mini-Xception cho Chó & Mèo

Lắp ráp 3 bảo bối của Học sâu vào chung một mạng (Mini-Xception):

- **Lỗi xử lý:** Thay vì dùng Conv2D, ta sử dụng toàn bộ SeparableConv2D để giảm chi phí tính toán.
- **Bình ổn hóa:** Mỗi bước tích chập đều kèm theo BatchNormalization để ổn định tốc độ học.
- **Thông suốt Đạo hàm:** Mỗi Khối (Block) được bắc cầu bởi một **Residual Connection** sử dụng Projection 1×1 .

Đánh giá Độ chính xác (Accuracy)

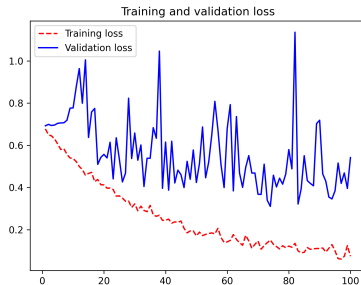
- Nhờ cấu trúc siêu ưu việt của SeparableConv và Residual, mô hình tự thiết kế của chúng ta đạt **Accuracy lên đến 90%** (Chỉ huấn luyện lại từ đầu với Data Augmentation, KHÔNG DÙNG Transfer Learning).
- Con số này đánh bại hoàn toàn mô hình Basic CNN (82%) ở Chương 8.



Hình 9.5: Accuracy của Mini-Xception

Đánh giá Mất mát (Loss)

- Biểu đồ Loss cho thấy đường Validation Loss đi xuống ổn định trong suốt quá trình.
- BatchNormalization giúp thuật toán hội tụ cực nhanh.
- Thiết kế này là tiêu chuẩn công nghiệp (Industry Standard) hiện nay trên Keras cho hầu hết mọi tác vụ Thị giác máy tính.



Hình 9.6: Loss của Mini-Xception

Để xây dựng một mô hình CNN xuất sắc từ con số 0, hãy tuân theo các nguyên tắc sau:

- Tổ chức Mô hình theo dạng Tháp Đặc trưng (Feature Pyramid). Lặp lại Khối (Blocks) thay vì tự nối từng Layer ngẫu nhiên (Quy tắc MHR).
- Sử dụng **Separable Convolution** thay cho Convolution tiêu chuẩn (Tính toán cực nhẹ).
- Kết hợp **Batch Normalization** để hệ thống hội tụ mượt mà không vấp ngã.
- Tạo các **Kết nối Thặng dư (Residual Connections)** để dòng chảy Gradient truyền xa xuyên suốt mạng sâu 100 tầng.